

Esame Programmazione di Sistema / API Programming

Data: 2023-09-04

QUESTIONS

Domanda 1

Attraverso un esempio pratico si illustri l'utilizzo dei Mutex nel linguaggio Rust. Qual è il meccanismo che consente il loro utilizzo in un contesto thread-safe?

Domanda 2

Si descriva il concetto di “ownership” in Rust e come contribuisca a prevenire errori come le race condition e i dangling pointer rispetto ad altri linguaggi come C++.

Domanda 3

Si dimostri come sia possibile implementare il polimorfismo attraverso i tratti. Si fornisca anche un esempio concreto che faccia riferimento ad almeno due strutture diverse.

Domanda 4

Una cache è una struttura dati generica, thread-safe, che consente di memorizzare coppie chiave/valore per un periodo non superiore ad una durata stabilita per ciascuna coppia. Nell'intervallo di validità associato alla coppia, richieste di lettura basate sulla chiave restituiscono il valore corrispondente, se presente. Trascorso tale periodo, eventuali richieste relative alla stessa chiave non restituiscono più il valore.

Poiché il numero di richieste in lettura può essere molto maggiore delle richieste in scrittura, è necessario fare in modo che le prime possano sovrapporsi temporalmente tra loro, mentre le seconde dovranno necessariamente essere con accesso esclusivo. Per evitare la saturazione della struttura, quando si eseguono operazioni di scrittura, si provveda ad eseguire un ciclo di pulizia, eliminando le eventuali coppie scadute.

Si implementi in Rust la `struct Cache<K: Eq + Hash, V>` dotata dei seguenti metodi:

- `pub fn new() -> Self` *// Crea una nuova istanza*
- `pub fn size(&self) -> usize` *// Restituisce il numero di coppie presenti nella mappa*

- `pub fn put(&self, k: K, v: V, d: Duration) -> ()` *// Inserisce la coppia k/v con durata pari a d*
- `pub fn renew(&self, k: &K, d: Duration) -> bool` *// Rinnova la durata dell'elemento rappresentato dalla chiave k; restituisce true se la chiave esiste e non è scaduta, altrimenti restituisce false*
- `pub fn get(&self, k: &K) -> Option<Arc<V>>` *// Restituisce None se la chiave k è scaduta o non è presente nella cache; altrimenti restituisce Some(a), dove a è di tipo Arc<V>*

Si ricordi che `Duration` è una struttura contenuta in `std::time`, che rappresenta una durata non negativa. Può essere sommato ad un valore di tipo `std::time::Instant` (che rappresenta un momento specifico nel tempo) per dare origine ad un nuovo `Instant`, collocato più avanti nel tempo.

SOLUTIONS

Soluzione Domanda 1

In Rust, un `Mutex` (Mutual Exclusion) serve a proteggere i dati condivisi, permettendo ad un solo thread per volta di accedervi. Quando un thread vuole leggere o modificare il dato, deve prima acquisire il *lock*. Se il lock è già tenuto da un altro thread, il thread corrente viene sospeso finché il lock non viene rilasciato. In Rust, il `Mutex` "contiene" i dati che protegge: non si può accedere ai dati senza aver prima ottenuto il lock (che restituisce una *guardia* `MutexGuard`).

Per condividere un `Mutex` tra più thread, è necessario racchiuderlo in un `Arc` (Atomic Reference Counting). L'`Arc` si occupa di gestire il ciclo di vita in sicurezza in un contesto multi-thread, permettendo di avere copie del puntatore (cloni) passate in modo *thread-safe* alle diverse closure in esecuzione concorrente. Il meccanismo che garantisce ciò a livello di sistema di tipi è l'implementazione del trait `Send` e `Sync` sia per `Arc` che per `Mutex`. `Mutex<T>` è `Sync` solo se `T`: `Send`.

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // Creiamo un contatore condiviso, protetto da Mutex e Arc
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter_clone = Arc::clone(&counter);

        let handle = thread::spawn(move || {
            // Blocchiamo il mutex. Se un altro thread ha già il
            // lock,
            // questo thread aspetterà'
            let mut num = counter_clone.lock().unwrap();
            *num += 1;
            // Il lock viene automaticamente rilasciato qui quando '
            // num' esce dallo scope (RAII)
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("Risultato finale: {}", *counter.lock().unwrap());
}
```

Soluzione Domanda 2

Il concetto di **ownership** (possesso) in Rust è il meccanismo principale per la gestione sicura della memoria, senza necessità di un garbage collector. Si basa su tre regole fondamentali:

1. Ogni valore in Rust ha un solo e unico proprietario (owner).
2. Può esserci un solo owner per volta per un determinato valore.
3. Quando l'owner esce di scope, il valore viene automaticamente deallocato (liberato).

Collegato all'ownership c'è il sistema di **borrowing** (prestito), gestito a tempo di compilazione dal *borrow checker*:

- È possibile avere uno o più riferimenti immutabili (**&T**) a un dato simultaneamente.
- **Oppure** è possibile avere un solo ed esclusivo riferimento mutabile (**&mut T**).
- I riferimenti devono sempre essere validi e non possono sopravvivere al proprietario del dato.

Queste regole prevengono la maggior parte degli errori tipici del C++:

- **Dangling pointers:** In C++, è possibile creare un puntatore a una variabile allocata sullo stack e ritornarlo, oppure deallocare un'area di memoria pur mantenendo in giro altri puntatori verso di essa (use-after-free). In Rust, il borrow checker verifica i *lifetime* per garantire che ogni riferimento non sopravviva all'owner dei dati, rendendo i dangling pointer impossibili (in *safe Rust*).
- **Race conditions (data races):** Una *data race* si verifica quando due thread accedono contemporaneamente allo stesso dato e almeno uno dei due accessi è una scrittura. In Rust, per modificare un dato condiviso serve un riferimento mutabile (**&mut T**), che per le regole del borrowing è garantito essere **esclusivo**. Se due thread tentassero di accedere senza un **Mutex** o uno strumento *thread-safe*, il compilatore bloccherebbe il programma per via della condivisione non sicura, in quanto si avrebbero più riferimenti allo stesso dato di cui uno mutabile, il che viola il prestito esclusivo, oppure perché il dato non implementa il trait **Sync**.

Soluzione Domanda 3

In Rust, il polimorfismo si ottiene principalmente attraverso i **Tratti** (Traits). Un tratto definisce un comportamento (metodi) condiviso, analogo a un'interfaccia in altri linguaggi (es. Java). Il polimorfismo può essere applicato a tempo di compilazione (static dispatch) usando i *generics* ed i *trait bounds*, o a tempo di esecuzione (dynamic dispatch) usando i *trait objects* tramite **dyn Trait**.

Di seguito un esempio di *dynamic dispatch*:

```

trait Shape {
    fn area(&self) -> f64;
}

struct Circle {
    radius: f64,
}

impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.radius * self.radius
    }
}

struct Rectangle {
    width: f64,
    height: f64,
}

impl Shape for Rectangle {
    fn area(&self) -> f64 {
        self.width * self.height
    }
}

// Funzione che riceve un vettore di oggetti polimorfici (Trait
// Object)
fn print_areas(shapes: &Vec<Box<dyn Shape>>) {
    for shape in shapes {
        println!("Area: {}", shape.area());
    }
}

fn main() {
    let circle = Circle { radius: 2.0 };
    let rect = Rectangle { width: 3.0, height: 4.0 };

    // Costruiamo una lista di tipi eterogenei che implementano "
    // Shape"
    let shapes: Vec<Box<dyn Shape>> = vec![
        Box::new(circle),
        Box::new(rect),
    ];

    print_areas(&shapes);
}

```

Soluzione Domanda 4

Il test richiede una `Cache` con letture contemporanee consentite (tramite `RwLock`) e scritture in esclusiva (ottenendo il lock mutabile su `RwLock`). In aggiunta, durante le scritture (`put` o `renew`) potremmo scorrere la cache per eliminare tutti gli elementi scaduti. I valori archiviati sono restituiti come `Arc<V>`, pertanto la `HashMap` conterrà dei valori incapsulati in `Arc<V>` più il `Instant` della scadenza.

```
use std::collections::HashMap;
use std::sync::{Arc, RwLock};
use std::time::{Duration, Instant};
use std::hash::Hash;

pub struct Cache<K: Eq + Hash, V> {
    // Usiamo RwLock per favorire i lettori simultanei.
    // Il valore memorizzato e' (Arc<V>, Scadenza (Instant)).
    map: RwLock<HashMap<K, (Arc<V>, Instant)>>,
}

impl<K: Eq + Hash + Clone, V> Cache<K, V> {
    pub fn new() -> Self {
        Cache {
            map: RwLock::new(HashMap::new()),
        }
    }

    pub fn size(&self) -> usize {
        let guard = self.map.read().unwrap();
        // Nota: non pulisce gli scaduti ma ne valuta l'esistenza in
        // base al tempo reale?
        // Il testo chiede "Restituisce il numero di coppie presenti
        // ".
        // Qui ci limitiamo alla dimensione fisica della HashMap.
        guard.len()
    }

    fn cleanup(&self, guard: &mut HashMap<K, (Arc<V>, Instant)>>) {
        let now = Instant::now();
        // Retain mantiene solo le entry che soddisfano il predicato
        guard.retain(|_, (_, exp)| *exp > now);
    }

    pub fn put(&self, k: K, v: V, d: Duration) {
        let mut guard = self.map.write().unwrap();
        // Per evitare saturazione, facciamo pulizia durante la
        // scrittura
        self.cleanup(&mut guard);
    }
}
```

```

        let expiration = Instant::now() + d;
        guard.insert(k, (Arc::new(v), expiration));
    }

    pub fn renew(&self, k: &K, d: Duration) -> bool {
        let mut guard = self.map.write().unwrap();
        self.cleanup(&mut guard);

        let now = Instant::now();
        if let Some((_, expiration)) = guard.get_mut(k) {
            if *expiration > now {
                *expiration = now + d;
                return true;
            }
        }
        false
    }

    pub fn get(&self, k: &K) -> Option<Arc<V>> {
        let guard = self.map.read().unwrap();
        if let Some((v_arc, expiration)) = guard.get(k) {
            if *expiration > Instant::now() {
                return Some(Arc::clone(v_arc));
            }
        }
        None
    }
}

```